

协程技术调研

从并发演进到底层原理 · 语言对比 · 实验验证

kyc

2026 年 6 月

计算机学院

大纲

1. 为什么需要协程 — 并发演进的三次让步
2. 协程是什么 — 有栈/无栈与底层机制
3. 语言怎么支持 — 按机制分组对比

1. 实验验证 — 协程 vs 多线程实测
2. 总结与选型 — 决策树与关键洞察

Module 1 · 为什么需要协程

一句话：协程不是让 CPU 计算更快的魔法，而是用更低的内存和调度成本处理大量等待中的 I/O 任务。

并发模型的三次让步

多进程 → 多线程

- 进程地址空间和页表开销大
- 线程共享地址空间，创建成本低

多线程 → 事件循环

- 默认栈 1MB，内核切换 μs 级
- C10K 下线程数撞内存/调度天花板

事件循环 → 协程

- epoll/kqueue/IOCP 复用连接到少量线程
- 但回调地狱：状态难维护，异常难传播
- 协程 = 事件循环的语法糖：线性代码 + 异步执行

四种并发模型对比

维度	进程	线程	事件循环	协程
调度方	OS 抢占	OS 抢占	用户态	用户态（协作）
切换开销	数十 μ s	1–5 μ s	10 ns 回调	10–100 ns
栈空间	MB 级	1 MB	无（堆闭包）	2–8 KB，可伸缩
编码风格	IPC 复杂	线性+锁	嵌套回调	线性+await
典型规模	百级	千级	万–十万	百万级
适合场景	强隔离	CPU 密集	底层网络库	I/O 密集、高并发

Table 1: 四种并发模型核心差异

Module 2 · 协程是什么

定义与心智模型

协程 = 可暂停和恢复的函数。普通函数必须一路执行到返回；协程可以在 `await/yield` 处挂起，保存执行状态，之后由调度器恢复。

// C++20 协程：顺序写法，异步执行

```
Task<Response> handle(Request req) {  
    auto user = co_await db.query(req.uid);    // 暂停点 ①  
    auto data = co_await rpc.fetch(user);      // 暂停点 ②  
    co_return render(data);  
}
```

关键认知：协程不是「更快」，是把隐式状态机变成显式由编译器生成。运行时复杂度不变，开发心智复杂度↓↓↓。

有栈协程 vs 无栈协程

有栈协程 (Stackful)

- 每个协程独立用户态栈
- 切换 = 保存/恢复寄存器 + SP
- 任意深度调用链都能 `yield`
- 代表: Go goroutine、Java 虚拟线程

优势: 普通函数可直接作为协程, 无「函数颜色」问题

无栈协程 (Stackless)

- 编译器把函数改写为状态机
- 局部变量存放在堆上 `frame`
- 只能在协程函数顶层 `await`
- 代表: C++20、Python、C#、Rust

优势: 内存更小 (字节级 vs KB 级), 编译器优化空间大

底层机制：两种切换实现

无栈协程：编译器生成状态机

```
co_await a; → case 0: ... suspend
co_await b; → case 1: ... suspend
co_return x; → case 2: return
```

- 编译器自动拆分函数，生成 `switch(state)` 跳转
- `frame` 仅几十几百字节，切换 $\approx$ 一次间接跳转
- C++20 三件套： `promise_type` / `awaiter` / `coroutine_handle`

有栈协程： **20** 行汇编切换

`switch_ctx:`

```
push rbp; push rbx; push r12-r15    ; 保存 callee-saved
mov [rdi], rsp                      ; 保存旧 SP
mov rsp, [rsi]                      ; 切到新栈
pop r15-r12; pop rbx; pop rbp       ; 恢复
ret                                  ; 跳到新协程上次 yield 处
```

- Go goroutine 初始栈 2KB，按需增长到 GB

- Java 虚拟线程由 JVM 调度，跑在 `carrier` 平台线程上

Module 3 · 语言怎么支持

总览：按机制分组

语言	关键字	类型	调度器	函数颜色	引入
Go	go/chan	有栈/对称	GMP 内建	否	1.0 (2012)
Java 21	Thread.ofVirtual	有栈/对称	JVM ForkJoin	否	21 (2023)
C++20	co_await	无栈/非对称	用户自带	是	2020
Python	async/await	无栈/非对称	单线程事件循环	是	3.5 (2015)
C#	async/await	无栈/非对称	线程池	是	5.0 (2012)
Rust	async/await	无栈/非对称	第三方 (tokio)	是	1.39 (2019)

Table 2: 主流语言协程能力对比（按有栈/无栈分组）

核心差异：有栈协程消除了「函数颜色」问题——Go 和 Java 21 的同步代码无需修改即可获得协程能力。

有栈协程：Go GMP + Java 虚拟线程

Go GMP 调度器

- **G** = goroutine（2KB 起步栈）
- **M** = OS 线程（受 GOMAXPROCS 限制）
- **P** = 调度上下文（本地运行队列）
- 网络 poller 集成 epoll/IOCP
- Go 1.14+ 基于信号的抢占式调度

Java 21 虚拟线程

- Virtual Thread = JVM 管理的有栈协程
- 跑在少量 **carrier** 平台线程上
- JDK 阻塞 API 已改造为 yield
- **最大卖点**：旧代码一行不改
-  **synchronized/JNI** 会 pin 到 carrier

共同点：普通同步代码无需 `async/await` 标记，运行时自动处理挂起和恢复。

无栈协程: C++20 / Python / C# / Rust

C++20: 只给机制, 不给运行时

- 编译器生成 `coroutine_handle`
- 标准库没有调度器、Task、IO
- 需要 `cppcoro` / `libcoro` / `asio` 等库

Python `asyncio`

- `async def` 返回 `coroutine` 对象
- 事件循环用 `epoll` 等等待 `fd` 就绪
- ⚠️ GIL 仍在, CPU 密集无收益

C# `async/await`: 开山鼻祖

- 编译器生成 `IAsyncStateMachine`
- `SynchronizationContext` 决定恢复线程
- C++20/Rust/Python 的 `async/await` 都借鉴 C# 5.0

Rust `async`: 零成本抽象

- 编译器生成 `Future` 状态机
- 生命周期规则让 `frame` 管理更复杂
- 生态靠 `tokio` / `async-std`

三层协同：编译器 × 运行时 × OS

层	职责	关键技术
编译器	识别 <code>async/await</code> ，改写为可暂停状态机	状态机 + <code>frame</code> 布局
运行时	调度协程、管理 <code>Task</code> 、对接 I/O	<code>work stealing</code> 、 <code>IO poller</code>
操作系统	提供非阻塞 I/O 多路复用	<code>epoll</code> / <code>kqueue</code> / <code>io_uring</code> / <code>IOCP</code>

Table 3: 协程机制的三层协作

运行时等待 `fd` 就绪后，再恢复对应协程。协程不是脱离操作系统独立存在的机制，而是编译器、运行时和 OS I/O 多路复用的协同产物。

Module 4 · 实验验证

实验设计

测试用例

1. **I/O 密集**: 10000 个并发任务, 每个 `sleep(0.1s)`
2. **CPU 密集**: 1000 个并发任务, 斐波那契($n=25$)
3. **上下文切换**: `yield / channel / sleep(0)` 量级测试

每项测试重复 5 次, 报告 $\text{mean} \pm \text{std}$

对比对象

- Python: `threading` vs `asyncio`
- Go: `goroutine`
- Java: 平台线程池 vs `Virtual Thread`

指标: 总耗时、峰值工作集、切换开销量级

测试机: i9-13900H (14C/20T), Win11, Python 3.12, Go 1.25, JDK 21

I/O 密集型结果

模型	10K I/O 耗时	相对基线	备注
Python threading	3.130±0.426 s	1.0×	OS 线程
Python asyncio	0.446±0.012 s	7.0×	单线程事件循环
Go goroutine	0.116±0.020 s	Go 内建	GMP 调度
Java platform	5.458±0.013 s	1.0×	200 固定线程池
Java virtual	0.235±0.173 s	23.2×	JDK 21 虚拟线程

Table 4: I/O 密集型实验结果（5 次重复）

关键发现

- 协程/虚拟线程对 OS 线程优势 7–23×
- Go/Java 虚拟线程能在 0.2 秒内完成 10000 个 sleep 任务
- 原因：用户态调度避免大量内核线程阻塞在等待态

CPU 密集型与切换开销

CPU 密集型 (fib(25))

模型	耗时
Python threading	9.521±1.591 s
Python asyncio	7.647±0.293 s
Go goroutine	0.039±0.010 s
Java platform	0.034±0.002 s
Java virtual	0.040±0.013 s

结论：协程不会加速 CPU 计算。Go/Java 的优势来自多核调度。

切换开销量级

模型	ns/次
Python threading	5000
Python asyncio	2494±144
Go goroutine	1220±150
Java virtual	750±320

注：各语言微基准不完全等价，仅用于量级比较。

内存使用对比

语言	内存 (MB)	测量方式	说明
Python threading	21.1±0.4	峰值工作集	10K I/O 任务
Python asyncio	14.1±0.5	峰值工作集	10K I/O 任务
Go goroutine	106.5	MemStats.Sys	10K goroutine
Java virtual	246	heap+nonHeap	10K 虚拟线程

Table 7: 各语言 I/O 并发任务内存使用（10K 任务）

注意：跨语言内存对比仅作参考——Python/Go/Java 使用不同的内存测量方法和运行时模型。Python 的优势在于其协程 frame 极小；Go/Java 的较高内存反映了运行时系统本身的开销。

实验洞察

三个关键发现

1. **I/O 密集时**: 协程对 OS 线程优势 7–23×, 主要来自「不进内核 + 不要 1MB 栈」
2. **CPU 密集时**: 协程**没有任何加速**——要并行还得多线程/多进程/GOMAXPROCS
3. **切换开销**: Go/Java 协程切换 μs 级, Python asyncio ms 级 (受事件循环开销影响)

三个易踩的坑

1. **阻塞调用毒化**: 在 async 里调用同步 `requests.get / synchronized` → 整个 worker 被钉死
2. **未结构化的并发**: 协程泄漏 = 内存泄漏 + 任务永不返回。用 `TaskGroup / errgroup / coroutineScope`
3. **误用做 CPU 加速**: 协程只省切换, 不省 CPU。CPU 瓶颈请上多核 / SIMD / GPU

Module 5 · 总结与选型

选型决策树

flowchart TD

```
Start["我的任务是?"] --> Q1{"瓶颈在?"}
Q1 -->|CPU| CPU["多线程 / 多进程 / GPU"]
Q1 -->|I/O| Q2{"并发量?"}
Q2 -->|"<1k"| Thread["OS 线程足够"]
Q2 -->|"1k-百万"| Q3{"语言生态?"}
Q3 -->|新项目, 性能优先| Go["Go: goroutine"]
Q3 -->|JVM 旧系统| Java["Java 21 虚拟线程"]
Q3 -->|Python 业务| Py["asyncio + uvloop"]
Q3 -->|系统级, 零开销| Cpp["C++20 / Rust async"]
```

经验法则：想最少改代码 → Java 21 / Go · 想最快上手 → Python asyncio · 想极致性能 → Rust tokio

总结

协程的本质：把等待态任务从昂贵的内核线程中解放出来，用编译器状态机、用户态调度器和 OS I/O 多路复用共同支撑高并发。

技术趋势：

- **有栈方向：**Go goroutine + Java 虚拟线程 → 强调普通同步代码的可扩展性
- **无栈方向：**C++20 / Rust / Python / C# → 强调精细控制和低内存
- **OS 层：**io_uring 正在改变 Linux 异步 I/O 的格局

AI 使用说明

1. AI 辅助梳理结构和术语校对，所有实验和代码由本人完成
2. 实验数据为本机实测（5 次重复），非 AI 编造
3. 对关键版本号、JEP/PEP 回查官方文档

参考资料

标准/提案

- ISO C++20: [dcl.fct.def.coroutine] · JEP 444: Virtual Threads (Java 21)
- PEP 492/525/530: Python coroutines & async generators

经典文献

- Conway M. [Design of a Separable Transition-Diagram Compiler](#), 1963
- Moura & Ierusalimschy [Revisiting Coroutines](#), TOPLAS 2009
- Nystrom [What Color is Your Function?](#), 2015

运行时实现

- Go runtime: runtime/proc.go (GMP) · OpenJDK Loom 项目
- libuv / tokio / asio 设计文档

课程信息

- 并行程序设计 · 计算机学院 · 2026 年 6 月